

Assignment 9 Instructions: Simple MVC GUI Application (Python + PySide6)

Purpose

The goal of this assignment is to demonstrate a **proof of concept (PoC)** for building a **desktop GUI application** using **Python**, **Qt Designer**, and **PySide6**, following the **Model–View–Controller (MVC)** architectural pattern. PoC here means a small app.

Learning Objectives

By completing this assignment, you will:

- Use **Qt Designer** to create a GUI
- Convert a .ui file into Python code or load it dynamically
- Apply the **MVC pattern** in a PySide6 application
- Connect UI events (signals) to application logic (slots)
- Structure a Python project using best practices

Technical Requirements

- Use Python
- Use PySide6
- Use Qt Designer
- Your app will be a cross-platform app and work on any OS, like Windows, macOS, or Linux

Application Description

Create a **simple desktop application** that:

- Accepts **user input** via the GUI
- Processes the input using a **Model**
- Displays output using a **View**
- Coordinates logic using a **Controller**

Example App Ideas (Choose One)

You may choose *one* of the following or propose an equivalent:

- A Counter app that lets user enter numbers and can increment / decrement a central counter
- A Simple calculator (just add and subtract). We did this before but remember: here the calculator has to be designed with MVC!
- Name formatter app: user enters a name/word as String. The app has features to capitalize or reverse the String.
- Temperature converter: lets user input a temperature. Pick one: do either F to C, or C to F. You don't have to do both. That is the PoC part. And remember: all input is String, so you have to grab the input and convert it to numeric data before you can use it in a calculation.

Model-View-Controller Design – Who Does What?

1. Model

The **Model**:

- Holds application data
- Performs business logic
- Has **no dependency on Qt widgets**

Examples

- Counter value
- Calculation logic
- Data validation

2. View

The **View**:

- Is designed using **Qt Designer**

- Contains only UI elements
- Emits signals for user actions
- Does **not** implement business logic

Requirements

- Create the UI visually in Qt Designer
- Save the file as view.ui
- Either:
 - Convert it using pyside6-uic (super easy!), **or**
 - Load it dynamically at runtime

3. Controller

The **Controller**:

- Connects signals from the View to logic in the Model
- Updates the View when the Model changes
- Acts as the “glue” of the application

Required Project Structure – this is your project road map! If you implement it like this you have an MVC design and are not sitting in front of an empty screen...

project_root/

```
|
|
|— main.py      # Application entry point
|— model.py     # Model logic
|— controller.py # Controller logic
|— view.ui      # Qt Designer UI file
|— view.py      # UI file converted to Python - if using pyside6-uic method
```

└─ README.md # Explanation of the project – **you can ask the AI to emit md or MarkDown code for you.**

Implementation Guidelines

Step 1: Design the UI

- Pick your app from the examples listed on page 2 or email me if you want to do something else. I will get right back to you with feedback.
- Open Qt Designer to create the ui file
- Create a simple layout, based on the app you picked. The minimum requirements are below, but if you are doing add and subtract, you will need 2 inputs, and so on.
 - At least one input widget (e.g., QLineEdit, QPushButton)
 - At least one output widget (e.g., QLabel)
- Save as view.ui

Step 2: Create the Model

- Store application state
- Implement methods to manipulate that state
- Do not import PySide6 in the model

Step 3: Create the Controller

- Load the UI
- Connect signals (e.g., button clicks)
- Call Model methods
- Update View widgets

Step 4: Create the Entry Point

- Initialize QApplication

- Create the Controller
- Show the main window
- Start the event loop

Functional Requirements Checklist

Uses Qt Designer

Follows MVC design pattern (separation)

Has working signal–slot connections

UI updates correctly based on model state

Runs without errors

Deliverables

Create a GitHub repo and add these files:

1. All .py source files
2. The .ui file
3. README.md including:
 - Application purpose
 - MVC explanation (brief)
 - How to run the app

README Example Structure - (Note that the symbols are the “markdown” and will format the document once you view it in your repo. This is good to know.).

Simple MVC PySide6 App

Description

A basic counter application demonstrating MVC architecture.

MVC Overview

- Model: Handles counter value
- View: GUI built in Qt Designer
- Controller: Connects buttons to model updates

How to Run

main.py

Grading Criteria

Item	Weight
MVC architecture correctness	35%
UI functionality	35%
Code clarity and organization	25%
README and documentation	5%

Notes

- Focus on **clean structure**, not advanced features
- Keep logic simple and readable